

Module 3: Semantic Analysis & Intermediate Code Generation

SDD, SDTS, Three-Address Code, Type Checking & Multi-D Arrays

Module III: Semantic Analysis & Intermediate Code Generation — Topics Covered

1. Syntax-Directed Definitions (SDD) & Attributes
2. S-Attributed vs. L-Attributed Definitions
3. Syntax-Directed Translation Schemes (SDTS)
4. Type Systems, Checking & Coercion
5. Three-Address Code (TAC): Quadruples & Triples
6. Multi-Dimensional Array Address Calculations

1. Syntax-Directed Definitions (SDD)

An **SDD** is a Context-Free Grammar augmented with attributes and semantic rules.

Attribute Category	Definition & Dependency Order	Evaluation Paradigm
Synthesized Attribute	Computed solely from the values of attributes at the child nodes in the parse tree: $A.s = f(Y_1.a, Y_2.b, \dots, Y_k.c)$.	Evaluated naturally during Bottom-Up Parsing (Post-order traversal).
Inherited Attribute	Computed from attributes of the parent node and/or sibling nodes: $Y_j.i = f(A.a, Y_1.b, \dots, Y_{j-1}.c)$.	Evaluated during Top-Down Parsing (Pre-order / In-order traversal).

S-Attributed vs. L-Attributed SDD Classification

S-Attributed SDD: Uses *exclusively synthesized attributes*. Can be evaluated directly in bottom-up LR parsers without building explicit trees.

L-Attributed SDD: Allows synthesized attributes AND inherited attributes, provided inherited attributes of Y_j depend only on A 's inherited attributes or attributes of siblings to the *left* ($Y_1, \dots, Y_{j-1}$). Evaluated in a single Depth-First Search (DFS) left-to-right pass.

2. Three-Address Code (TAC) Representations

Three-Address Code instructions have at most one operator on the RHS: $x = y \text{ op } z$.

Quadruple
(op, arg1, arg2, result)

Triple
(op, arg1, arg2) - index is result

Indirect Triple
Array of pointers to triples

FIGURE 3.1: COMPARISON OF THREE-ADDRESS CODE DATA STRUCTURES

3. Multi-Dimensional Array Address Calculations

Consider an array $A[d_1][d_2]$ with element size w and base address Base . Let lower bounds be $\text{low}_1, \text{low}_2$:

Row-Major Order (C/C++, Java)

$$\text{Address}(A[i][j]) = \text{Base} + \left((i - \text{low}_1) \times n_2 + (j - \text{low}_2) \right) \times w$$

Column-Major Order (Fortran, MATLAB)

$$\text{Address}(A[i][j]) = \text{Base} + \left((j - \text{low}_2) \times n_1 + (i - \text{low}_1) \right) \times w$$

BIT Mesra Numerical: 2D Array TAC Generation

Problem: Generate Three-Address Code for the assignment `x = A[i][j]` where A is a 10×20 integer array ($w = 4$ bytes, $\text{low}_1 = 1, \text{low}_2 = 1$) stored in Row-Major order.

Derivation:

```
// Address formula: Base + ((i - 1)*20 + (j - 1))*4
t1 = i - 1
t2 = t1 * 20
t3 = j - 1
t4 = t2 + t3
t5 = t4 * 4
t6 = A[t5]      // Array dereference with offset t5
x = t6
```